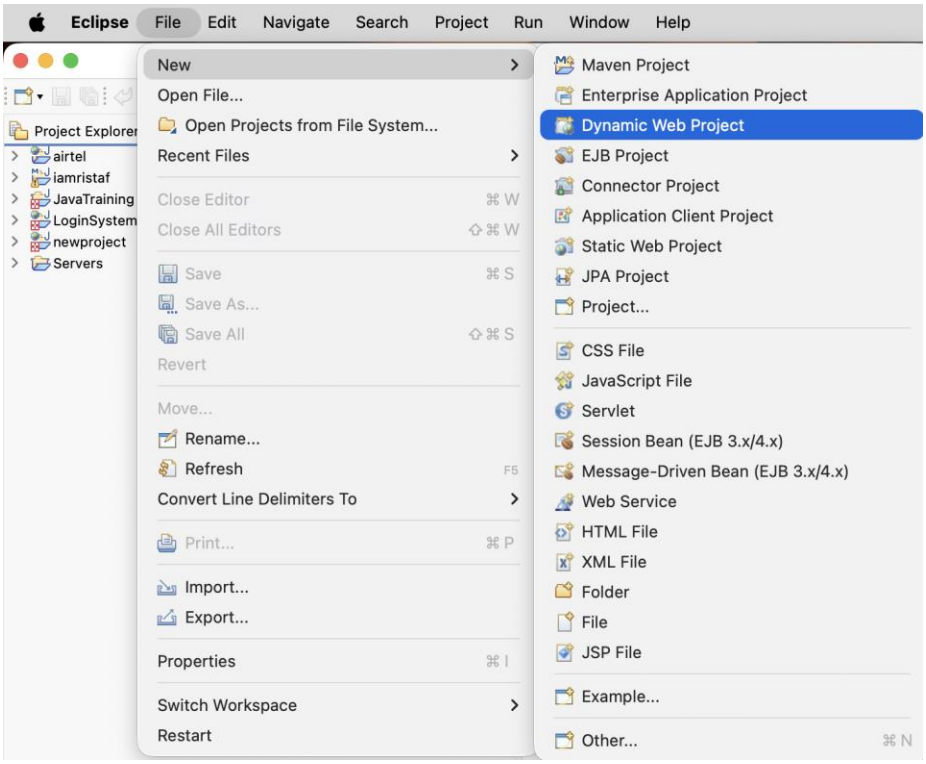
Program 1: Write a program to configure the Apache Tomcat in Eclipse.

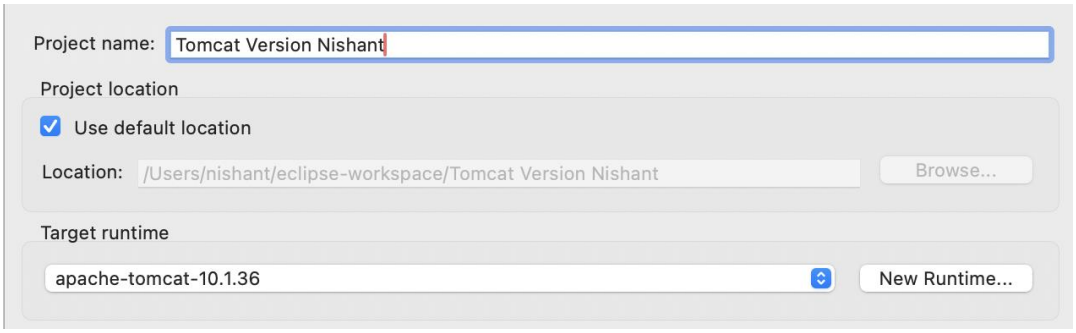
Step 1: Download & Install Tomcat

1. Go to Official Website https://tomcat.apache.org
2. Download the latest version (eg. Tomcat 9 or 10)
3. Extract the zip file to a folder

Step 2: Create a new Dynamic Web Project



Step 3: Then create a new Dynamic Web project in which select Target Runtime as apache-tomcat-10.1.36 or another version of tomcat as required. Then click on next and let Eclipse download Tomcat as in Dynamic Web Project.



Sample Code (Index.jsp)

<%@ page language="java" contentType="text/html; charset=UTF-8"

```
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <title>Tomcat Test</title>
</head>
<body>
    <h1>Hello, Apache Tomcat is Configured Successfully! </h1>
</body>
</html>
```

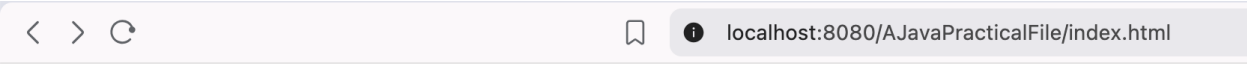
Program 2: Write a program to Create URL-Rewriting, Cookies, Http Session And Hidden Field.

Program Implementation Using Cookies

```
// CookieServlet.java

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

public class CookieServlet extends HttpServlet {

   public void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {

      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      Cookie ck = new Cookie("username", "Nishant");

      response.addCookie(ck);

      out.println("<h3>Cookie has been set!</h3>");

      out.println("<a href='GetCookie'>Get Cookie</a>");

   }

}


// GetCookie.java

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

public class GetCookie extends HttpServlet {

   public void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {

      PrintWriter out = response.getWriter();

      Cookie[] cookies = request.getCookies();
```

```
        if (cookies != null) {
            for (Cookie c : cookies) {
                out.println("Name: " + c.getName() + " Value: " + c.getValue());
            }
        }
    }
}
```

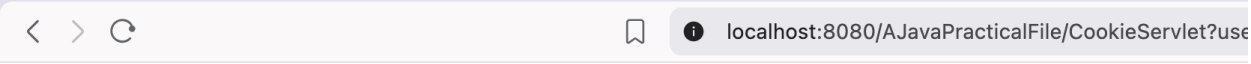
Output after running:



Enter Your Name

Name: Nishant Submit

After hitting submit button:

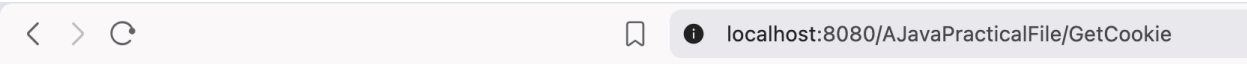


Served at: /AJavaPracticalFile

Cookie has been set!

Get Cookie

After clicking Get Cookie we get:



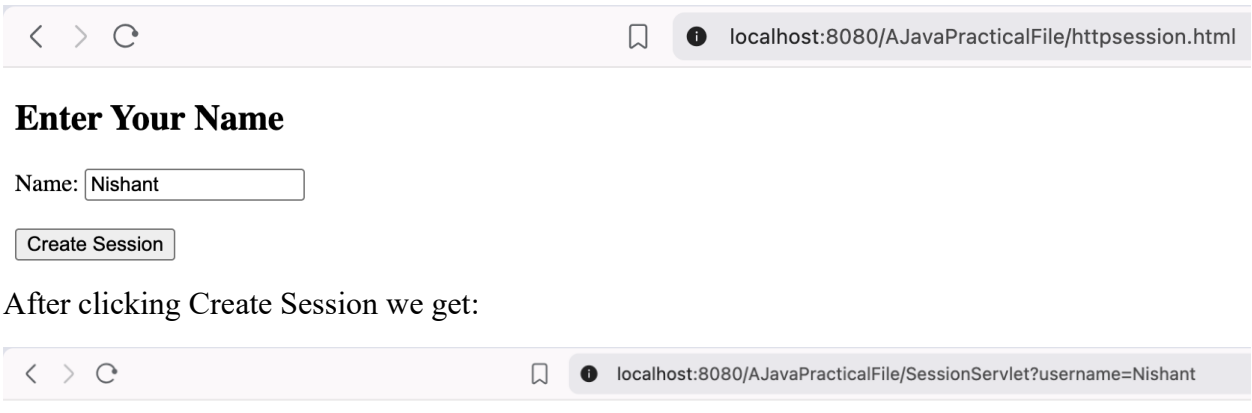

```
Served at: /AJavaPracticalFileName: username Value: Nishant
```

Program Implementation Using Http Session

```
// SessionServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SessionServlet extends HttpServlet {
```

```
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {
        PrintWriter out = response.getWriter();
        HttpSession session = request.getSession();
        session.setAttribute("user", "Nishant");
        out.println("Session created. <a href='GetSession'>Get Session</a>");
    }
}

// GetSession.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class GetSession extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession(false);
        if (session != null) {
            out.println("User: " + session.getAttribute("user"));
        }
    }
}
```

Output:



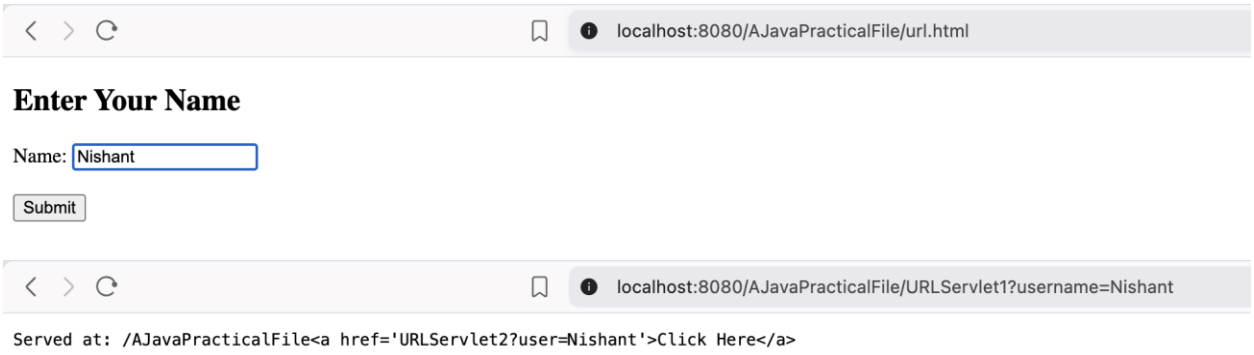
After clicking Create Session we get:

Program Implementation using URL-Rewriting

```
// URLServlet1.java

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;


public class URLServlet1 extends HttpServlet {

   public void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {

      PrintWriter out = response.getWriter();


      String name = "Nishant";

      out.println("<a href='URLServlet2?user=" + name + "'>Click Here</a>");

   }

}



// URLServlet2.java

import java.io.*;

import javax.servlet.*;
```

```
import javax.servlet.http.*;


public class URLServlet2 extends HttpServlet {
   public void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {
      PrintWriter out = response.getWriter();
      String user = request.getParameter("user");
      out.println("Welcome " + user);
   }
}
```

Output

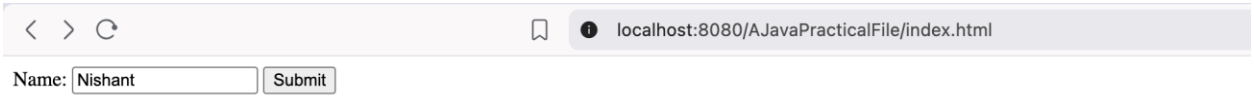


Program Implementation using Hidden Fields

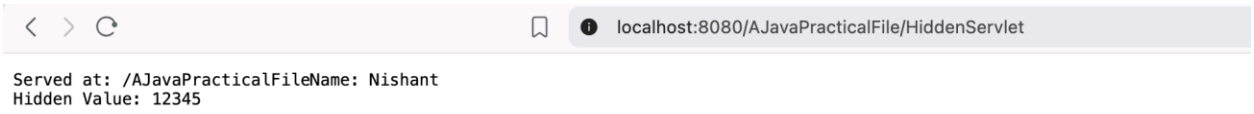
```
<!-- index.html -->
<form action="HiddenServlet" method="post">
   Name: <input type="text" name="username">
   <input type="hidden" name="hiddenValue" value="12345">
   <input type="submit" value="Submit">
</form>


// HiddenServlet.java
import java.io.*;
```

```
import javax.servlet.*;
import javax.servlet.http.*;
public class HiddenServlet extends HttpServlet {
   public void doPost(HttpServletRequest request, HttpServletResponse response) throws
IOException {
      PrintWriter out = response.getWriter();
      String name = request.getParameter("username");
      String hidden = request.getParameter("hiddenValue");
      out.println("Name: " + name);
      out.println("Hidden Value: " + hidden);
   }
}
```

Output:



After clicking submit, we get:



Program 3: Write a program to create JSP with JSTL

Step 1: Download JSTL jar files and place them inside: WEB-INF/lib/

Step 2: Create JSP file (index.jsp)

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<head>
    <title>JSTL Example</title>
</head>
<body>
<h2>JSTL Example Program</h2>
<c:set var="name" value="Nishant" />
<p>Name: <c:out value="${name}" /></p>
<c:if test="${name == 'Nishant'}">
    <p>Welcome, Nishant!</p>
</c:if>
<h3>Numbers from 1 to 5:</h3>
<c:forEach var="i" begin="1" end="5">
    <p>Number: <c:out value="${i}" /></p>
</c:forEach>
```

</body>

</html>

Output:

JSTL Example Program

Name: Nishant

Welcome, Nishant!

Numbers from 1 to 5:

Number: 1

Number: 2

Number: 3

Number: 4

Number: 5

Program 4: Write a program to analyze the use of JDBC with JSP and Servlet

Step 1: Create a dynamic web project and in MySQL create a database using following commands

```
CREATE DATABASE student_db;

USE student_db;

CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(50),
    course VARCHAR(50)
);
```

Step 2: Create a DBConnection.java file in any package with Servlet file

```
import java.sql.Connection;
import java.sql.DriverManager;
public class DBConnection {
    public static Connection getConnection() {
        Connection con = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/student_db",
                "root",
                "root"
            );
        } catch (Exception e) {
            e.printStackTrace();
```

```
        }
        return con;
    }
}

//StudentServlet.java (in same package as DBConnection.java)
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
@WebServlet("/addStudent")
public class StudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");
        try {
            Connection con = DBConnection.getConnection();
            String query = "INSERT INTO students(name, email, course) VALUES(?,?,?)";
            PreparedStatement ps = con.prepareStatement(query);
            ps.setString(1, name);
            ps.setString(2, email);
            ps.setString(3, course);
            int i = ps.executeUpdate();
```

```
            if (i > 0) {
                response.sendRedirect("success.jsp");
            } else {
                response.sendRedirect("error.jsp");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Step 3: Create all JSP Files in webapp folder

//index.jsp

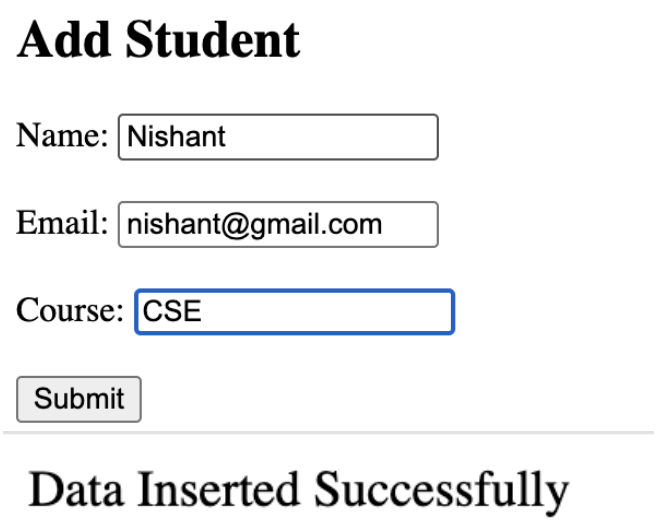
```
<!DOCTYPE html>
<html>
<head>
    <title>Student Form</title>
</head>
<body>
<h2>Add Student</h2>
<form action="addStudent" method="post">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="text" name="email"><br><br>
    Course: <input type="text" name="course"><br><br>
    <input type="submit" value="Submit"></form>
</body>
</html>
```

```
//success.jsp
<!DOCTYPE html>
<html>
<head>
    <title>Success</title>
</head>
<body>
<h2>Data Inserted Successfully!</h2>
<a href="display.jsp">View Students</a>
</body>
</html>

//error.jsp
<!DOCTYPE html>
<html>
<head>
    <title>Error</title>
</head>
<body>
<h2>Error inserting data!</h2>
</body>
</html>

//display.jsp
<%@ page import="java.sql.*" %>
<%@ page import="yourpackage.DBConnection" %>
```

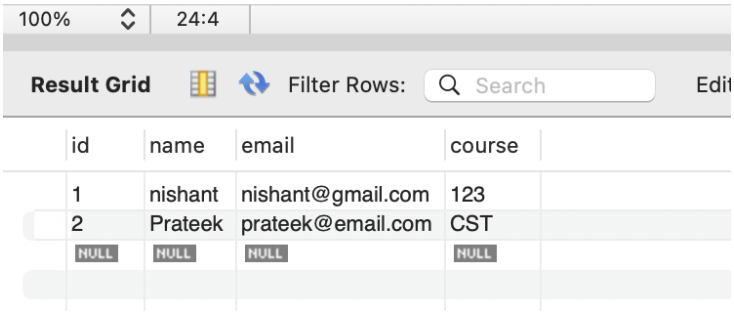
```
<!DOCTYPE html>
<html>
<head>
    <title>Student List</title>
</head>
<body>
<h2>Student Records</h2>
<table border="1">
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Course</th>
</tr>
<%
    try {
        Connection con = DBConnection.getConnection();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM students");
        while (rs.next()) {
%>
<tr>
    <td><%= rs.getInt("id") %></td>
    <td><%= rs.getString("name") %></td>
    <td><%= rs.getString("email") %></td>
    <td><%= rs.getString("course") %></td>
</tr>
```

```
<%
        }
    } catch (Exception e) {
        out.println(e);
    }
%>
</table>
</body>
</html>
```

OUTPUT:



```
1    use student_db;
2
3    show tables;
4    select * from students;
```



id	name	email	course
1	nishant	nishant@gmail.com	123
2	Prateek	prateek@email.com	CST
NULL	NULL	NULL	NULL

Program 5: Write a program to create simple struts

Step 1: Create a Maven Project and firstly add dependencies in POM file

//pom file

<project xmlns="http://maven.apache.org/POM/4.0.0"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.struts</groupId>

<artifactId>StrutsMavenApp</artifactId>

<version>1.0</version>

<packaging>war</packaging>

<dependencies>

<dependency>

<groupId>org.apache.struts</groupId>

<artifactId>struts2-core</artifactId>

<version>2.5.33</version>

</dependency>

<dependency>

<groupId>commons-fileupload</groupId>

<artifactId>commons-fileupload</artifactId>

<version>1.5</version>

</dependency>

<dependency>

<groupId>commons-io</groupId>

<artifactId>commons-io</artifactId>

<version>2.11.0</version>

</dependency>

</dependencies>

<build>

<finalName>StrutsMavenApp</finalName>

</build>

</project>

Step 2: Now create a struts.xml file

//struts.xml <!DOCTYPE struts PUBLIC

"-//Apache Software Foundation//DTD Struts Configuration 2.5//EN"

"http://struts.apache.org/dtds/struts-2.5.dtd">

<struts>

<package name="default" namespace="/" extends="struts-default">

<action name="hello" class="com.action.HelloAction">

<result name="success">/success.jsp</result>

</action>

</package>

</struts>

Step 3: In web.xml file write following code:

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"

version="3.1">

<filter>

<filter-name>struts2</filter-name>

<filter-class>

org.apache.struts2.dispatcher.filter.StrutsPrepareAndExecuteFilter

</filter-class>

</filter>

<filter-mapping>

<filter-name>struts2</filter-name>

<url-pattern>/*</url-pattern>

</filter-mapping>

</web-app>

Step 4: We will create a HelloAction.java class and add following code in that

```
package com.action;
public class HelloAction {
private String name;
public String execute() {
return "success";
}
public String getName() { return name; }
public void setName(String name) { this.name = name; }
}
```

Step 5: Now We will create jsp files in maven project index.jsp and success.jsp

//index.jsp

```
<%@ taglib prefix="s" uri="/struts-tags" %>

<html>

<body>

<h2>Enter Your Name</h2>

<s:form action="hello">

<s:textfield name="name" label="Name"/>

<s:submit value="Submit"/>

</s:form>

</body>

</html>

//success.jsp

<%@ taglib prefix="s" uri="/struts-tags" %>

<html>

<body>

<h2>Welcome, <s:property value="name"/>!</h2>

</body>

</html>
```

OUTPUT

Program 6: Write a program to create validation with struts

Step 1: Create a maven project and add following dependecies in pom file

//pom file

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>Struts2ValidationApp</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>
<dependencies>
<dependency>
<groupId>org.apache.struts</groupId>
<artifactId>struts2-core</artifactId>
<version>2.5.30</version>
</dependency>
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>
<version>4.0.1</version>
<scope>provided</scope>
</dependency>
</dependencies>
</project>
```

Step 2: Create Xml files that are struts.xml and web.xml

//struts.xml

```
<!DOCTYPE struts PUBLIC
"-//Apache Software Foundation//DTD Struts Configuration 2.5//EN"
"http://struts.apache.org/dtds/struts-2.5.dtd">
<struts>
<package name="default" namespace="/" extends="struts-default">
<action name="register" class="com.example.action.RegisterAction">
<result name="success">success.jsp</result>
<result name="input">index.jsp</result>
</action>
</package>
</struts>
```

//web.xml

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
<filter>
<filter-name>struts2</filter-name>
<filter-class>org.apache.struts2.dispatcher.filter.StrutsPrepareAndExecuteFilter</filter-class>
</filter>
<filter-mapping>
<filter-name>struts2</filter-name>
<url-pattern>/*</url-pattern>
</filter-mapping>
```

```
</web-app>
```

Step 3: Create a RegisterAction.java class for validation

```
//RegisterAction.java
package com.example.action;
import com.opensymphony.xwork2.ActionSupport;
public class RegisterAction extends ActionSupport {
private String name;
private String email;
public String execute() {
return SUCCESS;
}
public void validate() {
if (name == null || name.trim().isEmpty()) {
addFieldError("name", "Name is required");
}
if (email == null || !email.contains("@")) {
addFieldError("email", "Valid email is required");
}
}
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
```

}

Step 4: Now create two jsp files for UI that are index.jsp and success.jsp

//index.jsp

```
<%@ taglib prefix="s" uri="/struts-tags" %>
<html>
<head><title>Register</title></head>
<body>
<h2>Registration Form</h2>
<s:form action="register">
<s:textfield name="name" label="Name"/>
<s:textfield name="email" label="Email"/>
<s:submit value="Submit"/>
</s:form>
</body>
</html>
```

//success.jsp

```
<%@ taglib prefix="s" uri="/struts-tags" %>
<html>
<head><title>Success</title></head>
<body>
<h2>Registration Successful!</h2>
Name: <s:property value="name"/><br/>
Email: <s:property value="email"/>
```

```
</body>

</html>
```

OUTPUT

Program 7: Write a program to implement HQL

Step 1: Setup a Maven Project with hibernate file and Pom file
Step 2: Create a class Student.java

```
//Student.java

import jakarta.persistence.*;

@Entity

@Table(name = "student")

public class Student {

   @Id

   @GeneratedValue(strategy = GenerationType.IDENTITY)

   private int id;

   private String name;

   private String course;

   public Student() {}

   public Student(String name, String course) {

      this.name = name;

      this.course = course;

   }

   public int getId() { return id; }

   public String getName() { return name; }

   public void setName(String name) { this.name = name; }

   public String getCourse() { return course; }

   public void setCourse(String course) { this.course = course; }

}
```

Step 3: Add Mapping in hibernate.cfg.xml

```
<mapping class="com.z_practical_7.Student"/>
```

Step 4: Use HQL in main App.java file

```
import org.hibernate.*;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;
import com.z_practical_7.Student;
import java.util.List;
public class MainApp {
    public static void main(String[] args) {
        // Create SessionFactory
        SessionFactory factory = new
Configuration().configure("hibernate.cfg.xml").buildSessionFactory();
        Session session = factory.openSession();
        Transaction tx = session.beginTransaction();
        // Insert sample data
        session.persist(new Student("Nishant", "Java"));
        session.persist(new Student("Rahul", "Python"));
        session.persist(new Student("Amit", "Hibernate"));


        tx.commit();
        // HQL Query
        session.beginTransaction();
        Query<Student> query = session.createQuery("from Student", Student.class);
        List<Student> list = query.list();
        // Display results
        for (Student s : list) {
            System.out.println(s.getId() + " " + s.getName() + " " + s.getCourse());
        }
        session.close();
```

```
        factory.close();
    }
}
```

OUTPUT

```
1 • use new;
2   select * from student;
```

100% 23:2

Result Grid Filter Rows: Search

id	course	name
1	Java	Nishant
2	Python	Rahul
3	Hibernate	Amit
NULL	NULL	NULL

Program 8: Write a program to implement One to Many mapping

Step 1: Create a Maven Project in Eclipse IDE and create a hibernate config file

//hibernate.cfg.xml

```
<?xml version='1.0' encoding='utf-8'?>

<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN" "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

<session-factory>

<property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>

<property name="hibernate.connection.url">jdbc:mysql://localhost:3306/test</property>

<property name="hibernate.connection.username">root</property>

<property name="hibernate.connection.password">12345678</property>

<property name="hibernate.dialect">

org.hibernate.dialect.MySQLDialect

</property>

<property name="hibernate.hbm2ddl.auto">update</property>

<property name="hibernate.show_sql">true</property>

<mapping class="com.z_practical_8.Department"/>

<mapping class="com.z_practical_8.Employee"/>

</session-factory>

</hibernate-configuration>
```

Step 2: Add Dependency for hibernate core and Mysql Connector in Pom file

Step 3: Create two java classes names Employee.java and Department.java

```
//Employee.java

package com.z_practical_8;

import jakarta.persistence.*;

@Entity

public class Employee {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private int id;

private String name;

@ManyToOne

@JoinColumn(name = "dept_id")

private Department department;

// Getters and Setters

public int getId() { return id; }

public void setId(int id) { this.id = id; }

public String getName() { return name; }

public void setName(String name) { this.name = name; }

public Department getDepartment() { return department; }

public void setDepartment(Department department) {

this.department = department;

}

}
```

```
//Department.java
package com.z_practical_8;
import jakarta.persistence.*;
import java.util.List;
@Entity
public class Department {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;
private String name;
@OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
private List<Employee> employees;
// Getters and Setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public List<Employee> getEmployees() { return employees; }
public void setEmployees(List<Employee> employees) { this.employees = employees; }
}
```

Step 4: Write following code in App.java file (main file)

```
//app.java
package com.z_practical_8;
```

```
import org.hibernate.*;
import org.hibernate.cfg.Configuration;
import java.util.Arrays;
public class App {
public static void main(String[] args) {
SessionFactory factory = new Configuration().configure().buildSessionFactory();
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
Department dept = new Department();
dept.setName("IT");
Employee e1 = new Employee();
e1.setName("Nishant");
e1.setDepartment(dept);
Employee e2 = new Employee();
e2.setName("Rahul");
e2.setDepartment(dept);
dept.setEmployees(Arrays.asList(e1, e2));
session.persist(dept);
tx.commit();
session.close();
factory.close();
System.out.println("Data Inserted Successfully!");
} }
```

OUTPUT

```
1    use test;
2 •  select * from Employee;
```

100% 23:2

Result Grid Filter Rows: Search

id	name	dept_id
1	Nishant	1
2	Rahul	1
NULL	NULL	NULL

```
1    use test;
2 •  select * from Department;
```

100% 25:2

Result Grid Filter Rows: Search

id	name
1	IT
NULL	NULL

Program 9: Write a program to implement Named Query

Step 1: Create a maven project in Eclipse and setup pom file and hibernate file

Step 2: Create a class Student.java

```
//student.java
import jakarta.persistence.*;
@Entity
@Table(name = "student")
@NamedQuery(
    name = "Student.findAll",
    query = "FROM Student"
)
public class Student {
    @Id
    private int id;
    private String name;
    private String course;
    // Getters and Setters
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
```

```
        this.name = name;
    }
    public String getCourse() {
        return course;
    }
    public void setCourse(String course) {
        this.course = course;
    }
}
```

Step 3: Add mapping in hinernate file

```
<mapping class="com.z_practicall_9.Student"/>
```

Step 4: Add following code App.java

```
//App.java

package com.z_practicall_9;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import org.hibernate.query.Query;

import java.util.List;

public class App {

public static void main(String[] args) {

SessionFactory factory = new Configuration()

.configure("hibernate.cfg.xml")

.addAnnotatedClass(Student.class)

.buildSessionFactory();
```

```
Session session = factory.openSession();
session.beginTransaction();
// Insert data
Student s1 = new Student();
s1.setId(1);
s1.setName("Amit");
s1.setCourse("Java");
Student s2 = new Student();
s2.setId(2);
s2.setName("Neha");
s2.setCourse("Python");
session.persist(s1);
session.persist(s2);
session.getTransaction().commit();
// Using Named Query
Query<Student> query = session.createNamedQuery("Student.findAll", Student.class);
List<Student> list = query.getResultList();
for (Student s : list) {
System.out.println(s.getId() + " " + s.getName() + " " + s.getCourse());
}
session.close();
factory.close();
}
```

}

OUTPUT

```
1 • use practical_9;
2 • select * from student;
```

100% 17:1

Result Grid Filter Rows: Search Edit:

id	course	name
1	Java	Amit
2	Python	Neha
NULL	NULL	NULL

Program 10: Write a program to create a Login Page using Spring MVC & ORM

Step 1: First, Create a Maven Project with Dynamic Web Content in Eclipse ID

Step 2: Add Some Java Classes like User.java, UserDAO.java, LoginController.java

//User.java

```
package com.login.model;

import javax.persistence.*;

@Entity

@Table(name = "users")

public class User {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private int id;

@Column(name = "username")

private String username;

@Column(name = "password")

private String password;

// Getters and Setters

public int getId() { return id; }

public void setId(int id) { this.id = id; }

public String getUsername() { return username; }

public void setUsername(String username) { this.username = username; }

public String getPassword() { return password; }

public void setPassword(String password) { this.password = password; }
```

```
}
```

//UserDAO.java

```
package com.login.dao;
import com.login.model.User;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
@Repository
public class UserDAO {
   @Autowired
   private SessionFactory sessionFactory;
   public User getUser(String username, String password) {
      Session session = sessionFactory.openSession();
      User user = null;
      try {
         String hql = "FROM User u WHERE u.username = :uname AND u.password = :pwd";
         user = (User) session.createQuery(hql)
                  .setParameter("uname", username)
                  .setParameter("pwd", password)
                  .uniqueResult();
      } finally {
         session.close();
      }
      return user;
   }
}
```

//LoginController.java

```
package com.login.controller;
import com.login.dao.UserDAO;
import com.login.model.User;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
import org.springframework.ui.Model;
@Controller
public class LoginController {
    @Autowired
    private UserDAO userDAO;
    // Show login page
    @GetMapping("/login")
    public String showLoginPage() {
        return "login";
    }
    // Handle login form submission
    @PostMapping("/login")
    public String processLogin(
            @RequestParam("username") String username,
            @RequestParam("password") String password,
            Model model) {
        User user = userDAO.getUser(username, password);
        if (user != null) {
            model.addAttribute("username", user.getUsername());
            return "welcome";
```

```
        } else {
            model.addAttribute("error", "Invalid username or password!");
            return "error";
        }
    }
}
```

Step 3: Now we will create dispatcher-servlet.xml, and make some changes in web.xml

//dispatcher-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<context:component-scan base-package="com.login" />
<!-- View Resolver -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/views/" />
<property name="suffix" value=".jsp" />
</bean>
<!-- DataSource -->
```

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="com.mysql.cj.jdbc.Driver" />
<property name="url" value="jdbc:mysql://localhost:3306/logindb" />
<property name="username" value="root" />
<property name="password" value="12345678" />
</bean>
<!-- Hibernate SessionFactory -->
<bean id="sessionFactory"
class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
<property name="dataSource" ref="dataSource" />
<property name="annotatedClasses">
<list>
<value>com.login.model.User</value>
</list>
</property>
<property name="hibernateProperties">
<props>
<prop key="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</prop>
<prop key="hibernate.show_sql">true</prop>
<prop key="hibernate.hbm2ddl.auto">update</prop>
</props>
</property>
```

```
</bean>

<!-- Transaction Manager -->

<bean id="transactionManager"

class="org.springframework.orm.hibernate5.HibernateTransactionManager">

<property name="sessionFactory" ref="sessionFactory" />

</bean>

</beans>
```

//web.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee

http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"

version="3.1">

<servlet>

<servlet-name>dispatcher</servlet-name>

<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

<load-on-startup>1</load-on-startup>

</servlet>

<servlet-mapping>

<servlet-name>dispatcher</servlet-name>

<url-pattern>/</url-pattern>

</servlet-mapping>
```

<welcome-file-list>

<welcome-file>login</welcome-file>

</welcome-file-list>

</web-app>

Step 4: Now we will create some jsp files inside views folder in WEB-INF folder

//login.jsp

<%@ page language="java" contentType="text/html; charset=UTF-8" %>

<!DOCTYPE html>

<html>

<head>

<title>Login Page</title>

</head>

<body>

<h2>Login</h2>

<form action="login" method="post">

<label>Username</label>

<input type="text" name="username" placeholder="Enter username" required />

<label>Password</label>

<input type="password" name="password" placeholder="Enter password" required />

<input type="submit" value="Login" />

</form>

```
</body>

</html>
```

//**error.jsp**

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>

<!DOCTYPE html>

<html>

<head><title>Login Failed</title>

</head>

<body>

<div class="box">

<h2>Login Failed</h2>

<p>${error}</p>

<a href="login">← Try Again</a>

</div>

</body>

</html>
```

//welcome.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>

<!DOCTYPE html>

<html>

<head><title>Welcome</title>

</head>

<body>
```

```
<div class="box">

<h2>Login Successful!</h2>

<p>Welcome, <strong>${username}</strong>! You are logged in.</p>

</div>

</body>

</html>
```

OUTPUT